

Figure 6: Summary of pipeline hazards

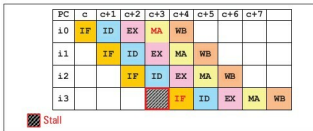


Figure 7: Preventing a pipeline hazard by stalling

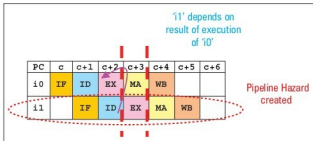


Figure 8: Dependency between instructions

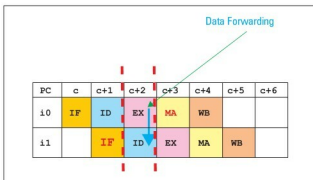


Figure 9: Data forwarding to prevent a pipeline hazard

this, let's use the same example as was discussed in the data hazards section:

ADD R1, R2, R3 //Instruction i0
MUL R4, R1, R2 //Instruction i1

Figure 8 depicts the dependency between instructions. The ADD instruction updates the value of register R1, and the MUL instruction uses the value of R1 as updated by the ADD instruction. Please note that the result of executing i0 will become available in the EX stage of the pipeline, and the MUL instruction will require reading the source operand in the ID stage. Thus, what we do here is, we forward the result of execution from the EX stage of i0 to the ID stage of i1. Thus, Instruction i1 need not wait for i0 to perform a write-back to register R1 in the WB stage. Figure 9 shows how this prevents the data hazard.

Now that we have understood pipelining and pipeline hazards, the question is, how can a developer exploit this feature to increase the throughput? In computer architecture, we use the term *clock cycles per instruction (CPI)* to identify the processor's performance. With pipelining, we can improve the CPI by exploiting instruction-level parallelism (ILP). This means executing more than one instruction simultaneously. For details on ILP, please refer to my article 'Dissecting Instruction-Level Parallelism' in the LFY May 2010 issue. Now, let's suppose we have an architecture that does not support any parallelism. In such a case, if we are able to fetch an instruction every cycle, we can have 5 instructions in the 5 pipeline stages at once (one instruction per stage). Thus, an instruction would complete Stage 5 in every clock cycle, and therefore on an average, we have one clock cycle per instruction (CPI = 1). With a single-issue processor, the best CPI attainable is 1. However, with multiple-issue processors, we may achieve even better CPI values. For example, a processor that issues two instructions per clock cycle can achieve a CPI of 0.5 when two instructions are completing every clock cycle.

In my next article, I will explain how the pipelining feature can be utilised by exploiting instruction scheduling and instruction level parallelism. **END**

Glossary

IF Instruction Fetch
ID Instruction Decode
EX Execute
MA Memory Access
WB Write Back
CPI Clock cycle per instruction
ILP Instruction Level Parallelism

References and further reading

1. Deepti Sharma. 'Exploring the dynamics of Pipelining,' Linux For You - Aug-2010
2. <http://www.cs.iastate.edu/~prabhu/Tutorial/PIPELINE/hazards.html>
3. http://en.wikipedia.org/wiki/Hazard_%28computer_architecture%29

By: Deepti Sharma

The author is a senior software engineer with a considerable amount of experience in the development of compilers and tools for embedded systems. She loves writing for LFY. She can be reached at deepti.acmet@gmail.com